Build in models

PCA

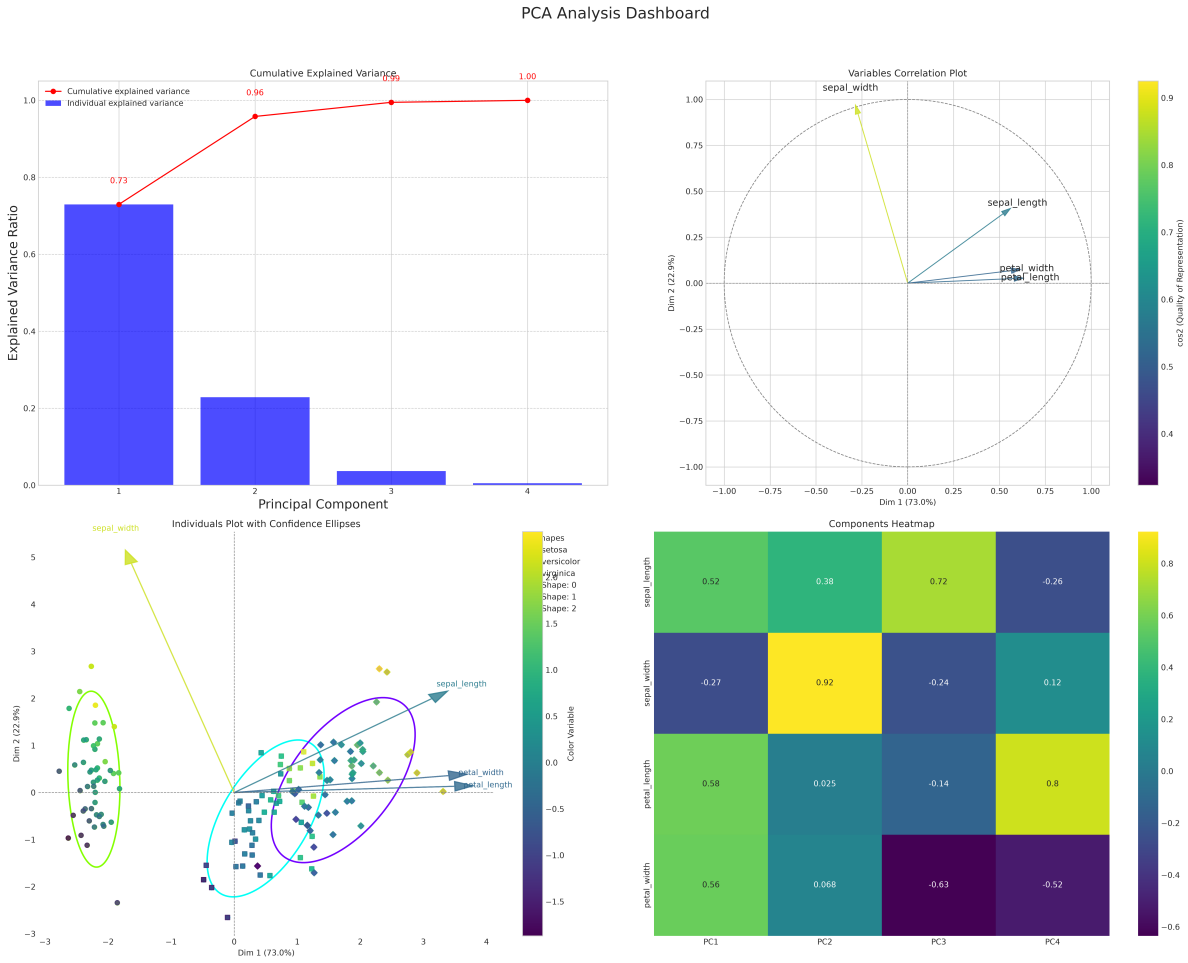
Python

```
from BayesForge import bf
import jax.numpy as jnp

m= bf()
m.data(m.load.iris(only_path=True))
m.data_on_model = dict(
    X=jnp.array(m.df.iloc[:,0:-2].values)
)
m.fit(m.models.pca(type="classic"), progress_bar=False) # or robust, sparse, classic, sparse

_ = m.models.pca.plot(
    X=m.df.iloc[:,0:-2].values,
    y=m.df.iloc[:,-2].values,
    feature_names=m.df.columns[0:-2],
    target_names=m.df.iloc[:,-1].unique(),
    color_var=m.df.iloc[:,0].values,
    shape_var=m.df.iloc[:,-2].values
)
```

```
jax.local_device_count 16
```



R

```r
library(BayesForge)
m <- importBF(platform = "cpu")
jnp <- reticulate::import("jax.numpy")
np <- reticulate::import("numpy")

d <- read.csv(m$load$iris(only_path = TRUE))
X <- as.matrix(d[, 1:4])  # all but the last two columns
m$data_on_model <- list(
  X = jnp$array(X)
)
m$fit(m$models$pca(type = "classic"), progress_bar = FALSE) # or robust, sparse, classic, spa
```

```
m$models$pca$plot(
  X = X,
  y = np$array(d$target),
  feature_names = names(d)[1:4],
  target_names = unique(d$species),
  color_var = np$array(d[, 1]),
  shape_var = np$array(d$target)
)
```

Julia

```
using PyBayesForge
using PythonCall

m = importBF(platform="cpu")
m.data(m.load.iris(only_path=true))
X = m.df.iloc[pybuiltins.slice(nothing), pybuiltins.slice(0, -2)].values  # all but the last
m.data_on_model = pydict(
    X=jnp.array(X)
)
m.fit(m.models.pca(type="classic"), progress_bar=false) # or robust, sparse, classic, sparse

@pyplot m.models.pca.plot(
    X=X,
    y=m.df.iloc[pybuiltins.slice(nothing), -2].values,
    feature_names=m.df.columns[pybuiltins.slice(0, -2)],
    target_names=m.df.iloc[pybuiltins.slice(nothing), -1].unique(),
    color_var=m.df.iloc[pybuiltins.slice(nothing), 0].values,
    shape_var=m.df.iloc[pybuiltins.slice(nothing), -2].values
)
```

Survival analysis

Piecewise-constant (piecewise-exponential) hazard model. Time is cut into fixed intervals; `import_time_even` builds the per-interval `death` and `exposure` matrices, and censoring is handled by `exposure` dropping to 0 once a subject leaves the risk set.

Python

```python
from BayesForge import bf
import jax.numpy as jnp

m = bf()
data_path = m.load.mastectomy(only_path=True)
m.data(data_path, sep=',')
m.df.metastasized = (m.df.metastasized.values == "yes").astype(jnp.int64)

# Discretise follow-up time into intervals and register the event indicator.
m.models.survival.import_time_even(
    m.df.time.values,
    m.df.event.values, interval_length=3
)

# Time-fixed (patient-level) covariates.
m.models.survival.import_covF(
    m.df.metastasized.values, ['metastasized']
)

# Time-varying covariates  Experimental feature
# m.models.survival.import_covV

# Censoring plot: one line per subject (follow-up length), red = censored,
# grey = event observed, dot = metastasized.
m.models.survival.plot_censoring(cov='metastasized')

# Priors (shown with their defaults, override before fit):
#   Baseline_rate ~ Gamma(*baseline_rate_prior)  -> near-flat, one rate/interval
#   Hazard_rate_<cov> ~ Normal(0, hazard_rate_prior_scale)  -> log-hazard effect
m.models.survival.baseline_rate_prior = (0.01, 0.01)
m.models.survival.hazard_rate_prior_scale = 10.0

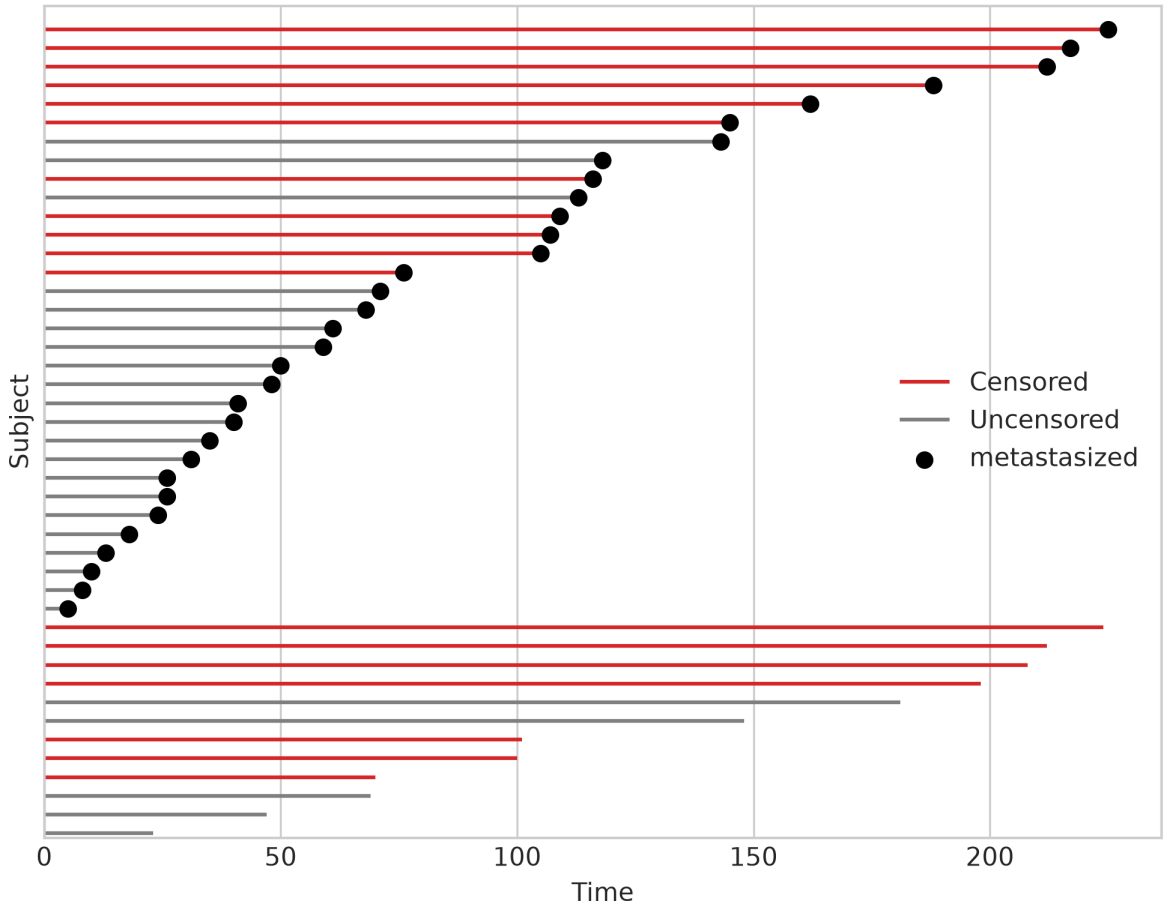
# 4 chains + target_accept_prob=0.99: the near-empty late baseline intervals
# make the geometry stiff, so a high acceptance target keeps mixing healthy.
m.fit(m.models.survival.model, num_samples=100, num_warmup=100,
      num_chains=4, target_accept_prob=0.99, progress_bar=False)

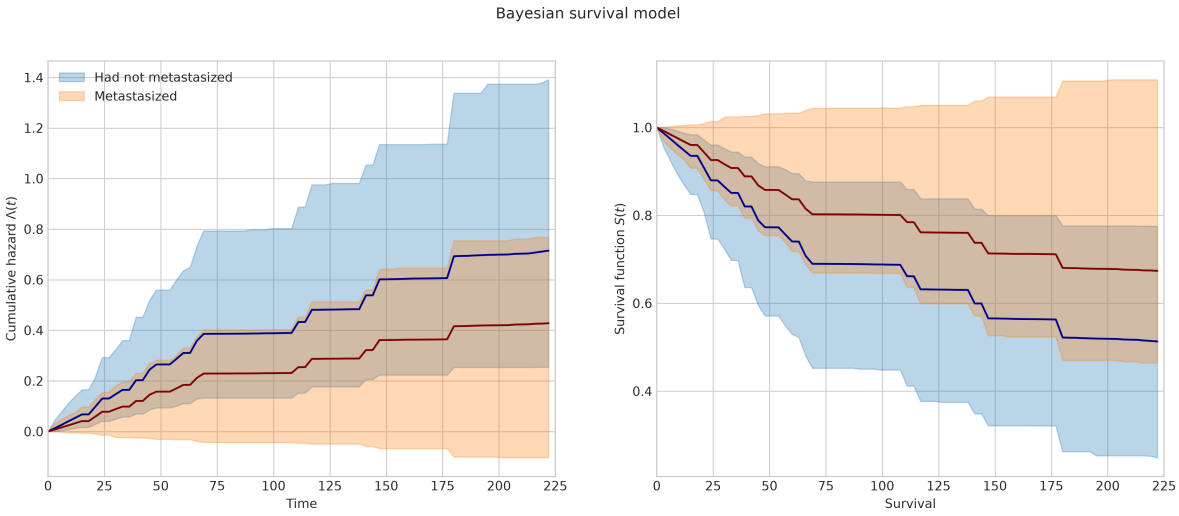
m.summary()

# Posterior cumulative hazard (left) and survival S(t)=exp(-Lambda(t)) (right)
```

```
# by covariate group. Effect size/direction: read beta (hazard ratio e^beta)
# from m.summary().
_ = m.models.survival.plot_surv(beta='Hazard_rate_metastasized')
```

```
jax.local_device_count 16
--------------------------------------------------------------------------------
Survival concern 44 individuals in 76 intervals.
26.0 individuals experienced the event.
--------------------------------------------------------------------------------
Covariates imported: ['metastasized']
Surv object now has 1 covariates: ['metastasized']
 This function is still in development. Use it with caution.
```





R

```r
library(BayesForge)
m <- importBF(platform = "cpu")
np <- reticulate::import("numpy")

data_path <- m$load$mastectomy(only_path = TRUE)
m$data(data_path, sep = ",")
met <- np$astype(np$equal(m$df$metastasized, "yes"), np$int64)
m$df$metastasized <- met

# Discretise follow-up time into intervals and register the event indicator.
m$models$survival$import_time_even(
  np$asarray(m$df$time),
  np$asarray(m$df$event), interval_length = 3L
)

# Time-fixed (patient-level) covariates.
m$models$survival$import_covF(
  met, list("metastasized")
)

# Time-varying covariates  Experimental feature
# m$models$survival$import_covV

# Censoring plot: one line per subject (follow-up length), red = censored,
```

```r
# grey = event observed, dot = metastasized.
m$models$survival$plot_censoring(cov = "metastasized")

# Priors (shown with their defaults, override before fit):
#   Baseline_rate ~ Gamma(*baseline_rate_prior)  -> near-flat, one rate/interval
#   Hazard_rate_<cov> ~ Normal(0, hazard_rate_prior_scale)  -> log-hazard effect
m$models$survival$baseline_rate_prior <- c(0.01, 0.01)
m$models$survival$hazard_rate_prior_scale <- 10.0

# 4 chains + target_accept_prob=0.99: the near-empty late baseline intervals
# make the geometry stiff, so a high acceptance target keeps mixing healthy.
m$fit(m$models$survival$model, num_samples = 100L, num_warmup = 100L,
      num_chains = 4L, target_accept_prob = 0.99, progress_bar = FALSE)

print(m$summary())

# Posterior cumulative hazard (left) and survival S(t)=exp(-Lambda(t)) (right)
# by covariate group. Effect size/direction: read beta (hazard ratio e^beta)
# from m$summary().
m$models$survival$plot_surv(beta = "Hazard_rate_metastasized")
```

Julia

```julia
using PyBayesForge
using PythonCall
np = pyimport("numpy")

m = importBF(platform="cpu")
data_path = pyconvert(String, m.load.mastectomy(only_path=true))
m.data(data_path, sep=",")
m.df.metastasized = np.astype(np.equal(m.df.metastasized.values, "yes"), np.int64)

# Discretise follow-up time into intervals and register the event indicator.
m.models.survival.import_time_even(
    m.df.time.values,
    m.df.event.values, interval_length=3
)

# Time-fixed (patient-level) covariates.
m.models.survival.import_covF(
    m.df.metastasized.values, ["metastasized"]
```

```
)

# Time-varying covariates  Experimental feature
# m.models.survival.import_covV

# Censoring plot: one line per subject (follow-up length), red = censored,
# grey = event observed, dot = metastasized.
@pyplot m.models.survival.plot_censoring(cov="metastasized")

# Priors (shown with their defaults, override before fit):
#   Baseline_rate ~ Gamma(*baseline_rate_prior)  -> near-flat, one rate/interval
#   Hazard_rate_<cov> ~ Normal(0, hazard_rate_prior_scale)  -> log-hazard effect
m.models.survival.baseline_rate_prior = (0.01, 0.01)
m.models.survival.hazard_rate_prior_scale = 10.0

# 4 chains + target_accept_prob=0.99: the near-empty late baseline intervals
# make the geometry stiff, so a high acceptance target keeps mixing healthy.
m.fit(m.models.survival.model, num_samples=100, num_warmup=100,
      num_chains=4, target_accept_prob=0.99, progress_bar=false)

println(m.summary())

# Posterior cumulative hazard (left) and survival S(t)=exp(-Lambda(t)) (right)
# by covariate group. Effect size/direction: read beta (hazard ratio e^beta)
# from m.summary().
@pyplot m.models.survival.plot_surv(beta="Hazard_rate_metastasized")
```

Gaussian Mixture Models

Python

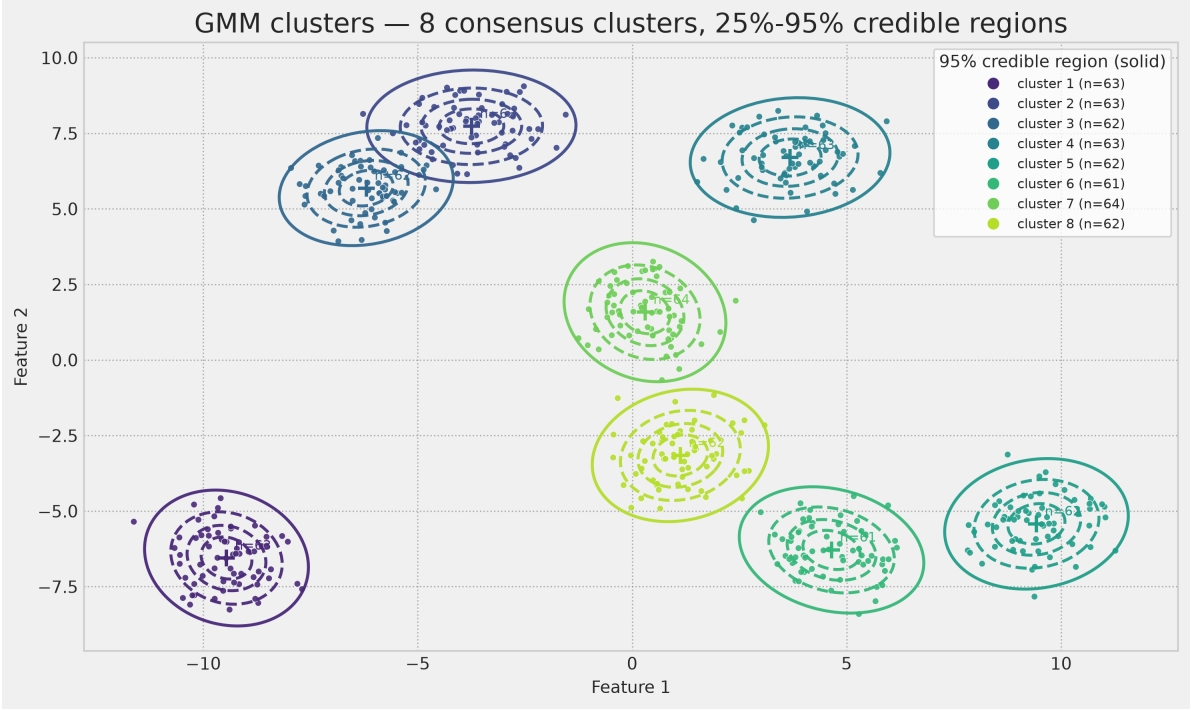
```
from BayesForge import bf
from sklearn.datasets import make_blobs
m = bf()

# Generate synthetic data
data, true_labels = make_blobs(
    n_samples=500, centers=8, cluster_std=0.8,
    center_box=(-10,10), random_state=101
)
```

```
m.data_on_model = {"data": data, "K": 8}
m.fit(m.models.gmm, progress_bar=False)  # MCMC over the mixture parameters
m.plot(X=data, sampler=m.sampler)    #   Experimental feature
```

```
jax.local_device_count 16
 This function is still in development. Use it with caution.
```

```
(<Figure size 3000x1800 with 1 Axes>,
 <Axes: title={'center': 'GMM clusters — 8 consensus clusters, 25%-95% credible regions'}, xl
```



R

```
library(BayesForge)
m <- importBF(platform = "cpu")
sk_datasets <- reticulate::import("sklearn.datasets")

# Generate synthetic data
output <- sk_datasets$make_blobs(
```

```
  n_samples = 500L, centers = 8L, cluster_std = 0.8,
  center_box = reticulate::tuple(-10, 10), random_state = 101L
)
data <- output[[1]]

m$data_on_model <- list(data = data, K = 8L)
m$fit(m$models$gmm, progress_bar = FALSE)  # MCMC over the mixture parameters
m$plot(X = data, sampler = m$sampler)      #   Experimental feature
```

Julia

```
using PyBayesForge
using PythonCall
m = importBF(platform="cpu")

# Generate synthetic data
sk_datasets = pyimport("sklearn.datasets")
output = sk_datasets.make_blobs(
    n_samples=500, centers=8, cluster_std=0.8,
    center_box=(-10, 10), random_state=101
)
data = output[0]

m.data_on_model = pydict(data=data, K=8)
m.fit(m.models.gmm, progress_bar=false)  # MCMC over the mixture parameters
@pyplot m.plot(X=data, sampler=m.sampler) #   Experimental feature
```

Dirichlet Process Mixture Models

Python

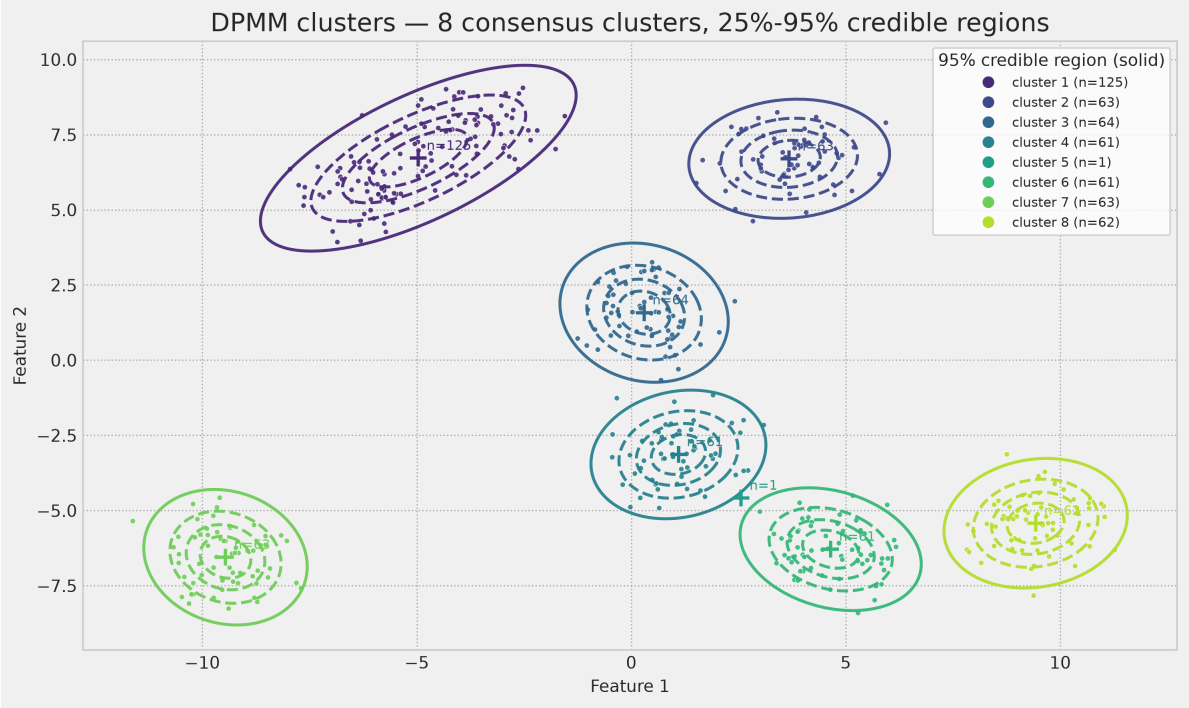
```
from BayesForge import bf
from sklearn.datasets import make_blobs
m = bf()

# Generate synthetic data
data, true_labels = make_blobs(
    n_samples=500, centers=8, cluster_std=0.8,
    center_box=(-10,10), random_state=101
```

```
)
# T = truncation level (max clusters). method='marginal' integrates out the
# cluster assignments (NUTS-friendly); 'latent' samples them explicitly.
# DPMM priors are already centred on the data mean/scale - no flag needed.
m.data_on_model = dict(data=data, T=10, method='marginal')
m.fit(m.models.dpmm, num_samples=500, num_warmup=500, num_chains=2,
      progress_bar=False)
m.plot(data, m.sampler)
```

```
jax.local_device_count 16
Model found 8 clusters.
```

```
(<Figure size 3000x1800 with 1 Axes>,
 <Axes: title={'center': 'DPMM clusters - 8 consensus clusters, 25%-95% credible regions'}, x
```



R

```r
library(BayesForge)
m <- importBF(platform = "cpu")
sk_datasets <- reticulate::import("sklearn.datasets")

# Generate synthetic data
output <- sk_datasets$make_blobs(
  n_samples = 500L, centers = 8L, cluster_std = 0.8,
  center_box = reticulate::tuple(-10, 10), random_state = 101L
)
data <- output[[1]]
# T = truncation level (max clusters). method='marginal' integrates out the
# cluster assignments (NUTS-friendly); 'latent' samples them explicitly.
# DPMM priors are already centred on the data mean/scale - no flag needed.
m$data_on_model <- list(data = data, T = 10L, method = "marginal")
m$fit(m$models$dpmm, num_samples = 500L, num_warmup = 500L, num_chains = 2L,
      progress_bar = FALSE)
m$plot(data, m$sampler)
```

Julia

```julia
using PyBayesForge
using PythonCall
m = importBF(platform="cpu")

# Generate synthetic data
sk_datasets = pyimport("sklearn.datasets")
output = sk_datasets.make_blobs(
    n_samples=500, centers=8, cluster_std=0.8,
    center_box=(-10, 10), random_state=101
)
data = output[0]
# T = truncation level (max clusters). method='marginal' integrates out the
# cluster assignments (NUTS-friendly); 'latent' samples them explicitly.
# DPMM priors are already centred on the data mean/scale - no flag needed.
m.data_on_model = pydict(data=data, T=10, method="marginal")
m.fit(m.models.dpmm, num_samples=500, num_warmup=500, num_chains=2,
      progress_bar=false)
@pyplot m.plot(data, m.sampler)
```

Network Models

Python

```python
from BayesForge import bf
import jax.numpy as jnp

# Setup device--------------------------------------------------
m = bf(platform='cpu')
# Simulate data ------------------------------------------------
N = 50
individual_predictor = m.dist.normal(0,1, shape = (N,1), sample = True)

kinship = m.dist.bernoulli(0.3, shape = (N,N), sample = True)
kinship = kinship.at[jnp.diag_indices(N)].set(0)

def sim_network(kinship, individual_predictor):
  # Intercept
  alpha = -8

  # SR
  sr = m.net.sender_receiver(individual_predictor, individual_predictor, s_mu = 0.4, r_mu = 

  # D
  DR = m.net.dyadic_effect(kinship, d_sd= 1, sample = True)

  return m.dist.bernoulli(logits = alpha + sr + DR, sample = True)


network = sim_network(m.net.mat_to_edgl(kinship), individual_predictor)

# Predictive model ---------------------------------------------
m.data_on_model = dict(
    network = network,
    dyadic_predictors = m.net.mat_to_edgl(kinship),
    focal_individual_predictors = individual_predictor,
    target_individual_predictors = individual_predictor
)

def model(network, dyadic_predictors, focal_individual_predictors, target_individual_predicto
    N_id = network.shape[0]
    alpha = m.dist.normal(0, 1, name="alpha")
```

```
    sr = m.net.sender_receiver(
        focal_individual_predictors,
        target_individual_predictors,
        s_mu=0.4, r_mu=-0.4
    )
    dr = m.net.dyadic_effect(dyadic_predictors, d_sd=1.0)
    m.dist.bernoulli(logits=alpha + sr + dr, obs=network)

m.fit(model, num_samples=500, num_warmup=500, num_chains=1, progress_bar=False)
m.summary()
```

```
jax.local_device_count 16
```

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd
alpha	-4.87	0.38	-5.50	-4.25	337.02	348.49	NaN	0.02	0.01
dr_L[0, 0]	1.00	0.00	1.00	1.00	500.00	500.00	NaN	0.00	NaN
dr_L[0, 1]	0.00	0.00	0.00	0.00	500.00	500.00	NaN	0.00	NaN
dr_L[1, 0]	0.02	0.40	-0.65	0.64	559.45	397.07	NaN	0.02	0.01
dr_L[1, 1]	0.91	0.11	0.69	1.00	324.87	349.05	NaN	0.01	0.01
...	...	...	...	...	...	...	...	...	...
sr_rf[48, 1]	-0.42	0.85	-1.97	0.86	656.48	351.73	NaN	0.03	0.05
sr_rf[49, 0]	-0.49	0.97	-2.09	0.95	1085.38	416.17	NaN	0.03	0.05
sr_rf[49, 1]	-0.42	0.90	-1.90	0.87	545.71	327.24	NaN	0.04	0.07
sr_sigma[0]	1.05	0.29	0.65	1.56	246.06	258.75	NaN	0.02	0.01
sr_sigma[1]	0.96	0.40	0.25	1.57	91.65	54.82	NaN	0.04	0.04

R

```
library(BayesForge)
jnp <- reticulate::import("jax.numpy")

# Setup device--------------------------------------------------
m <- importBF(platform = "cpu")
# Simulate data ------------------------------------------------
N <- 50L
individual_predictor <- m$dist$normal(0, 1, shape = reticulate::tuple(N, 1L), sample = TRUE)

kinship <- m$dist$bernoulli(0.3, shape = reticulate::tuple(N, N), sample = TRUE)
```

```r
diag <- jnp$diag_indices(N)
kinship <- kinship$at[diag[[1]], diag[[2]]]$set(0L)

sim_network <- function(kinship, individual_predictor) {
  alpha <- -8
  sr <- m$net$sender_receiver(individual_predictor, individual_predictor, s_mu = 0.4, r_mu =
  DR <- m$net$dyadic_effect(kinship, d_sd = 1, sample = TRUE)
  m$dist$bernoulli(logits = alpha + sr + DR, sample = TRUE)
}

network <- sim_network(m$net$mat_to_edgl(kinship), individual_predictor)

m$data_on_model <- list(
  network = network,
  dyadic_predictors = m$net$mat_to_edgl(kinship),
  focal_individual_predictors = individual_predictor,
  target_individual_predictors = individual_predictor
)

model <- function(network, dyadic_predictors, focal_individual_predictors, target_individual
  alpha <- m$dist$normal(0, 1, sample = TRUE)
  sr <- m$net$sender_receiver(focal_individual_predictors, target_individual_predictors, s_mu
  dr <- m$net$dyadic_effect(dyadic_predictors, d_sd = 1.0)
  m$dist$bernoulli(logits = alpha + sr + dr, obs = network)
}

m$fit(model, num_samples = 500L, num_warmup = 500L, num_chains = 1L, progress_bar = FALSE)
m$summary()
```

Julia

```julia
using PyBayesForge
using PythonCall

# Setup device-----------------------------------------------
m = importBF(platform="cpu")
# Simulate data ---------------------------------------------
N = 50
individual_predictor = m.dist.normal(0, 1, shape=(N, 1), sample=true)

kinship = m.dist.bernoulli(0.3, shape=(N, N), sample=true)
```

```
kinship = kinship.at[jnp.diag_indices(N)].set(0)

function sim_network(kinship, individual_predictor)
    alpha = -8
    sr = m.net.sender_receiver(individual_predictor, individual_predictor, s_mu=0.4, r_mu=-0
    DR = m.net.dyadic_effect(kinship, d_sd=1, sample=true)
    return m.dist.bernoulli(logits=alpha + sr + DR, sample=true)
end

network = sim_network(m.net.mat_to_edgl(kinship), individual_predictor)

m.data_on_model = pydict(
    network=network,
    dyadic_predictors=m.net.mat_to_edgl(kinship),
    focal_individual_predictors=individual_predictor,
    target_individual_predictors=individual_predictor
)

function model(network, dyadic_predictors, focal_individual_predictors, target_individual_pre
    alpha = m.dist.normal(0, 1, name="alpha")
    sr = m.net.sender_receiver(focal_individual_predictors, target_individual_predictors, s_r
    dr = m.net.dyadic_effect(dyadic_predictors, d_sd=1.0)
    m.dist.bernoulli(logits=alpha + sr + dr, obs=network)
end

m.fit(model, num_samples=500, num_warmup=500, num_chains=1, progress_bar=false)
m.summary()
```

Network Visualization

Python

```
from BayesForge import bf
import jax.numpy as jnp
import numpy as np
import pandas as pd

# Setup device-----------------------------------------------
m = bf(platform='cpu')
N = 50
```

```
individual_predictor = m.dist.normal(0, 1, shape=(N, 1), sample=True)

kinship = m.dist.bernoulli(0.3, shape=(N, N), sample=True)
kinship = kinship.at[jnp.diag_indices(N)].set(0)

def sim_network(kinship, individual_predictor):
    alpha = -4
    sr = m.net.sender_receiver(individual_predictor, individual_predictor, s_mu=0.4, r_mu=-0
    DR = m.net.dyadic_effect(kinship, d_sd=1, sample=True)
    return m.dist.bernoulli(logits=alpha + sr + DR, sample=True)

network = sim_network(m.net.mat_to_edgl(kinship), individual_predictor)
adj = m.net.edgl_to_mat(network)

# Individual characteristics of the nodes ----------------------------------
rng = np.random.default_rng(1)
nodes = pd.DataFrame({
    "id": [f"n{i}" for i in range(N)],
    "predictor": np.asarray(individual_predictor).ravel(),
    "sex": rng.choice(["female", "male"], N),
    "group": rng.choice(["A", "B", "C"], N),
    "age": rng.integers(1, 30, N),
    "rank": rng.uniform(0, 1, N),
})

# Interactive network: each node feature drives one visual channel
m.net.viz(
    nodes, adj,
    col_id="id",
    col_size="predictor",                                    # node size
    col_color="group", color_node=("#1b9e77", "#d95f02"),    # node fill colour
    col_stroke="age",                                        # border width
    col_strokeColor="rank", color_stroke=("white", "black"), # border colour
    col_shape="sex", shapes=("circle", "square"),            # node shape
)
```

```
jax.local_device_count 16
```

```
NetworkView(path=out/NetExplorer.html, 648320 bytes)
```

R

```r
library(BayesForge)
jnp <- reticulate::import("jax.numpy")
np <- reticulate::import("numpy")

m <- importBF(platform = "cpu")
N <- 50L
individual_predictor <- m$dist$normal(0, 1, shape = reticulate::tuple(N, 1L), sample = TRUE)

kinship <- m$dist$bernoulli(0.3, shape = reticulate::tuple(N, N), sample = TRUE)
diag <- jnp$diag_indices(N)
kinship <- kinship$at[diag[[1]], diag[[2]]]$set(0L)

sim_network <- function(kinship, individual_predictor) {
  alpha <- -4
  sr <- m$net$sender_receiver(individual_predictor, individual_predictor, s_mu = 0.4, r_mu =
  DR <- m$net$dyadic_effect(kinship, d_sd = 1, sample = TRUE)
  m$dist$bernoulli(logits = alpha + sr + DR, sample = TRUE)
}

network <- sim_network(m$net$mat_to_edgl(kinship), individual_predictor)
adj <- m$net$edgl_to_mat(network)

nodes <- data.frame(
  id = paste0("n", 0:(N - 1)),
  predictor = as.vector(individual_predictor),
  sex = sample(c("female", "male"), N, replace = TRUE),
  group = sample(c("A", "B", "C"), N, replace = TRUE),
  age = sample(1:29, N, replace = TRUE),
  rank = runif(N, 0, 1)
)

m$net$viz(
  nodes, adj,
  col_id = "id",
  col_size = "predictor",
  col_color = "group", color_node = c("#1b9e77", "#d95f02"),
  col_stroke = "age",
  col_strokeColor = "rank", color_stroke = c("white", "black"),
  col_shape = "sex", shapes = c("circle", "square")
)
```

Julia

```
using PyBayesForge
using PythonCall
np = pyimport("numpy")
pd = pyimport("pandas")

m = importBF(platform="cpu")
N = 50
individual_predictor = m.dist.normal(0, 1, shape=(N, 1), sample=true)

kinship = m.dist.bernoulli(0.3, shape=(N, N), sample=true)
kinship = kinship.at[jnp.diag_indices(N)].set(0)

function sim_network(kinship, individual_predictor)
    alpha = -4
    sr = m.net.sender_receiver(individual_predictor, individual_predictor, s_mu=0.4, r_mu=-0
    DR = m.net.dyadic_effect(kinship, d_sd=1, sample=true)
    return m.dist.bernoulli(logits=alpha + sr + DR, sample=true)
end

network = sim_network(m.net.mat_to_edgl(kinship), individual_predictor)
adj = m.net.edgl_to_mat(network)

nodes = pd.DataFrame(pydict(
    id=[string("n", i) for i in 0:(N-1)],
    predictor=np.asarray(individual_predictor).ravel(),
    sex=rand(["female", "male"], N),
    group=rand(["A", "B", "C"], N),
    age=rand(1:29, N),
    rank=rand(N)
))

@pyplot m.net.viz(
    nodes, adj,
    col_id="id",
    col_size="predictor",
    col_color="group", color_node=("#1b9e77", "#d95f02"),
    col_stroke="age",
    col_strokeColor="rank", color_stroke=("white", "black"),
    col_shape="sex", shapes=("circle", "square")
)
```